MODULE THREE

BUILDING YOUR TOOL

From Concept to Code: Your MVP in 7 Days

The moment of truth. You've validated your idea. You've planned your features. Now it's time to build something real. This module transforms you from planner to builder, from dreamer to creator.

Welcome to the Builder's Journey

The Critical Crossroads

This is where most aspiring entrepreneurs fail. Not because they lack ideas—you've already proven yours has merit in Module 2. Not because they lack planning—your validation sprint gave you a clear roadmap. They fail here, at the building stage, because of three fatal mistakes:

1. **Analysis Paralysis:** Spending months researching the "perfect" tech stack instead of shipping
2. **Feature Creep:** Adding "just one more thing" until the MVP becomes a bloated monster
3. **Perfectionism:** Polishing pixels while competitors capture your market

Module 3 is designed to eliminate these failure modes completely. We're going to build your Minimum Viable Product in **7 days or less**. Not 7 months. Not 7 weeks. *7 days.*

How? By following a proven framework that's already helped thousands of founders

Welcome to the Builder's Journey

The Critical Crossroads

This is where most aspiring entrepreneurs fail. Not because they lack ideas—you've already proven yours has merit in Module 2. Not because they lack planning—your validation sprint gave you a clear roadmap. They fail here, at the building stage, because of three fatal mistakes:

1. **Analysis Paralysis:** Spending months researching the "perfect" tech stack instead of shipping
2. **Feature Creep:** Adding "just one more thing" until the MVP becomes a bloated monster
3. **Perfectionism:** Polishing pixels while competitors capture your market

Module 3 is designed to eliminate these failure modes completely. We're going to build your Minimum Viable Product in **7 days or less**. Not 7 months. Not 7 weeks. *7 days.*

How? By following a proven framework that's already helped thousands of founders ship their first SaaS tool. You're not going to build the next Salesforce. You're going to build a focused, functional tool that solves one problem extremely well—and generates

your first dollar of revenue.

WHAT THIS MODULE DELIVERS

By the end of Module 3, you will have:

- A **functional web application** that solves your validated problem
- A **deployed product** accessible via your own domain name
- A **database backend** that stores user data securely
- A **payment integration** ready to accept your first customer (covered in Module 4, prepared here)
- A **testing checklist** ensuring your tool works across devices and browsers
- A **launch plan** with specific hour-by-hour tactics for maximum visibility

THE AI-ENHANCED ADVANTAGE

Why AI Changes Everything

This isn't your 2020 micro-SaaS playbook. The AI revolution has fundamentally transformed what's possible for solo founders and small teams. Tools that would have required machine learning PhDs and six-figure AWS bills can now be built in an afternoon using API calls.

Example: A traditional "Resume Analyzer" might parse PDFs and match keywords—useful but basic. An **AI-Enhanced Resume Analyzer** can:

- Provide specific improvement suggestions for each bullet point
- Rewrite sections to be more achievement-focused
- Tailor content to specific job descriptions
- Score resumes against ATS (Applicant Tracking Systems)
- Generate cover letters automatically

Same problem space. 10x more value. Built in the same timeframe.

Throughout this module, whenever we discuss a tool category, we'll show you both the traditional implementation and the AI-enhanced version. You'll learn when AI adds genuine value (frequently) and when it's unnecessary complexity (occasionally).

The Builder's Dilemma: Two Paths Diverge

Before we dive into building, you face a crucial decision. There are two fundamentally different approaches to building your micro-SaaS tool, and the path you choose will determine everything from your timeline to your long-term scalability.

PATH A: THE NO-CODE FOUNDER

Best For: Business-First Builders

Who This Is For: You're a marketer, designer, consultant, or domain expert.

The Builder's Dilemma: Two Paths Diverge

Before we dive into building, you face a crucial decision. There are two fundamentally different approaches to building your micro-SaaS tool, and the path you choose will determine everything from your timeline to your long-term scalability.

PATH A: THE NO-CODE FOUNDER

Best For: Business-First Builders

Who This Is For: You're a marketer, designer, consultant, or domain expert. You understand your customers deeply but don't have a programming background. You want to validate and monetize quickly without learning to code.

Primary Tools:

- **Bubble.io:** Visual programming for complex web apps
- **Softr:** Turn Airtable databases into customer-facing apps
- **Webflow:** Design-first website builder with CMS capabilities
- **Airtable + Zapier:** Database + automation for simpler tools

Advantages:

- Ship in days, not months
- Visual interfaces match your mental model
- Built-in hosting and deployment
- Less that can break
- Focus 100% on business problems, not technical problems

Limitations:

- Monthly platform costs ($29-99+)
- Less customization for complex features
- Dependent on platform's roadmap
- Harder to integrate AI APIs (but possible)
- May need to rebuild if you scale significantly

Realistic Timeline: 3-5 days from start to launch

PATH B: THE LOW-CODE BUILDER

Best For: Technical Tinkerers

Who This Is For: You have basic HTML/CSS knowledge or are willing to learn. You're comfortable following tutorials and debugging error messages. You want complete control and minimal ongoing costs.

Primary Tools:

- **HTML/CSS/JavaScript:** The foundation of the web
- **Supabase:** Backend-as-a-service (database + auth + storage)
- **Vercel/Netlify:** Free hosting with instant deployment
- **OpenAI API/Anthropic API:** AI capabilities via simple API calls

Advantages:

- Near-zero monthly costs (free tiers go far)
- Complete customization freedom
- Easy AI integration via APIs
- Valuable technical skills you'll use forever
- Own your entire stack

Limitations:

- Steeper learning curve upfront
- More pieces to connect
- You're responsible for all debugging
- Requires basic coding knowledge

Realistic Timeline: 5-7 days from start to launch (first time)

CHOOSING YOUR PATH: A DECISION FRAMEWORK

Answer these questions honestly:

CHOOSING YOUR PATH: A DECISION FRAMEWORK

Answer these questions honestly:

Path Selection Quiz

1. **Have you ever written HTML or any programming code?**
   Yes → +1 point for Path B | No → +1 point for Path A
2. **Do you enjoy troubleshooting technical problems?**
   Yes → +2 points for Path B | No → +2 points for Path A
3. **Is your tool relatively simple** (form → processing → result)?
   Yes → +1 point for either | No/Complex workflows → +2 points for Path A
4. **Are you willing to invest 10-20 hours learning coding basics?**
   Yes → +2 points for Path B | No → +2 points for Path A
5. **Is $30-100/month in platform costs acceptable?**
   Yes → +1 point for Path A | No/Want to minimize costs → +2 points for Path B
6. **Will your tool heavily use AI features?**
   Yes → +2 points for Path B | No → Neutral

Results:

- If Path A scored higher: Start with Lesson 3.1A (No-Code Track)
- If Path B scored higher: Start with Lesson 3.1B (Low-Code Track)

If tied: Start with Path A for speed, graduate to Path B for your second tool

> *The Hybrid Approach*
>
> Many successful founders start with Path A to validate and generate initial revenue, then rebuild with Path B once they have paying customers and proven demand. There's no shame in this—it's strategic. Your first version doesn't need to be your forever version.

MODULE STRUCTURE: CHOOSE YOUR ADVENTURE

This module is structured to support both paths:

- **Lesson 3.1A:** No-Code MVP (Bubble.io focus)
- **Lesson 3.1B:** Low-Code MVP (HTML/JS/Supabase focus) ← *Detailed in this document*
- **Lesson 3.2:** Adding Essential Features (applies to both)
- **Lesson 3.3:** Testing & Launch (applies to both)

The remainder of this comprehensive module will focus on **Path B: The Low-Code Builder** approach, as it offers the most flexibility for AI integration and long-term scalability. If you're following Path A, the companion "No-Code Builder's Guide" provides equivalent step-by-step instructions using Bubble.io and related platforms.

> *Important: No-Code Builders*
>
> If you selected Path A, you should switch to the "Module 3A: No-Code Builder's Edition" document. The technical instructions that follow assume basic coding

knowledge. Don't worry—both paths reach the same destination, and you can always switch later.

Lesson 3.1B: Building Your Low-Code MVP

THE FOUNDATION: UNDERSTANDING YOUR TECH STACK

Before writing a single line of code, let's understand what we're building with and why. Your tech stack is the combination of technologies that work together to make your tool function. Think of it like a recipe—each ingredient serves a specific purpose.

Your Micro-SaaS Tech Stack

We're using what's called a "JAMstack" architecture—JavaScript, APIs, and

Lesson 3.1B: Building Your Low-Code MVP

THE FOUNDATION: UNDERSTANDING YOUR TECH STACK

Before writing a single line of code, let's understand what we're building with and why. Your tech stack is the combination of technologies that work together to make your tool function. Think of it like a recipe—each ingredient serves a specific purpose.

Your Micro-SaaS Tech Stack

We're using what's called a "JAMstack" architecture—JavaScript, APIs, and Markup. It's:

- **Fast:** Static files load instantly
- **Secure:** No server means fewer attack vectors
- **Scalable:** Can handle traffic spikes effortlessly
- **Cheap:** Most services have generous free tiers

COMPONENT 1: THE FRONTEND (WHAT USERS SEE)

Technologies: HTML, CSS, JavaScript

This is your user interface—everything your customers see and interact with. We're keeping it intentionally simple:

- **HTML** provides the structure (forms, buttons, text)
- **CSS** provides the styling (colors, layouts, fonts)
- **JavaScript** provides the interactivity (button clicks, form validation, API calls)

> *Why This Stack?*
>
> **Why not React/Vue/Angular?** These modern frameworks are powerful but add unnecessary complexity for MVP micro-SaaS tools. You can always upgrade later. For now, vanilla JavaScript gives you everything you need with zero build process.
>
> **Why not WordPress?** WordPress is great for content sites, not web applications. You'd spend more time fighting the CMS than building your tool.
>
> **Why not a backend language like Python/PHP?** Because we're using serverless architecture. The backend is handled by Supabase (more on this next), so you don't need to manage servers.

COMPONENT 2: THE BACKEND (WHERE DATA LIVES)

Technology: Supabase

Supabase is an open-source Firebase alternative. In non-technical terms, it's a "Backend-as-a-Service" (BaaS) that gives you:

- **Database:** PostgreSQL database to store all your data
- **Authentication:** User signups, logins, password resets—all handled
- **Storage:** File uploads (images, PDFs, etc.)

- **API:** Automatically generated endpoints to read/write data

> *Why Supabase?*
>
> - **Generous Free Tier:** 500MB database, 1GB storage, 50,000 monthly active users
> - **SQL Power:** Real database, not just a JSON store
> - **Real-time Features:** Built-in WebSocket support for live updates
> - **Open Source:** If you outgrow it, you can self-host
> - **Excellent Documentation:** Clear examples for common use cases

COMPONENT 3: THE HOSTING (WHERE YOUR SITE LIVES)

Technology: Vercel or Netlify

These platforms host your frontend files (HTML, CSS, JS) and make them accessible via your domain. They're specifically designed for static sites and offer:

- **Free SSL certificates** (the https:// padlock)
- **Global CDN** (your site loads fast worldwide)
- **Instant deployment** (push code, it goes live in seconds)

COMPONENT 3: THE HOSTING (WHERE YOUR SITE LIVES)

Technology: Vercel or Netlify

These platforms host your frontend files (HTML, CSS, JS) and make them accessible via your domain. They're specifically designed for static sites and offer:

- **Free SSL certificates** (the https:// padlock)
- **Global CDN** (your site loads fast worldwide)
- **Instant deployment** (push code, it goes live in seconds)
- **Automatic backups** (version history for all changes)

> *Vercel vs. Netlify: Which to Choose?*
>
> Honest answer: both are excellent. For this course, we'll use **Vercel** because:
>
> - Slightly more generous free tier
> - Better serverless function support (useful for advanced features)
> - Created by the Next.js team (if you ever want to upgrade)
>
> But if you prefer Netlify's interface or have existing projects there, it works identically.

COMPONENT 4: THE AI LAYER (OPTIONAL BUT POWERFUL)

Technology: OpenAI API or Anthropic Claude API

If your tool benefits from AI capabilities, you'll integrate one of these APIs. An API is simply a way for your application to send requests to another service and receive responses.

When to Add AI

Add AI if your tool involves:

- Text generation (emails, social posts, product descriptions)
- Content analysis (resume scoring, grammar checking, sentiment analysis)
- Summarization (TL;DR generation, meeting notes, article digests)
- Translation or rewriting
- Question answering based on documents
- Creative work (story ideas, business names, taglines)

Skip AI if your tool:

- Performs calculations or data transformations (unit converters, calculators)
- Generates from templates (invoice generators, QR codes)
- Manipulates images/media without understanding content
- Simply displays or organizes information

PUTTING IT ALL TOGETHER: HOW THE STACK WORKS

Here's the flow when a user interacts with your tool:

1. **User visits your domain** → Vercel serves the HTML/CSS/JS files
2. **User clicks a button or submits a form** → JavaScript captures this action
3. **JavaScript makes an API call** → Either to Supabase (for data) or OpenAI (for AI)

4. **The service processes the request** → Database query runs or AI generates response
5. **Response returns to JavaScript** → Your code receives the data
6. **JavaScript updates the page** → User sees the result

> *The Beauty of This Architecture*
>
> Notice what's missing? You're not managing servers. You're not worrying about scaling infrastructure. You're not configuring databases. Each service handles its own domain, and you simply connect them together. This is why micro-SaaS has become so accessible—the hard parts are now managed services.

Your First Build: A Complete Walkthrough

Theory is important, but let's get practical. We're going to build a simple but functional tool from scratch. This example demonstrates every concept you'll need for your own

Your First Build: A Complete Walkthrough

Theory is important, but let's get practical. We're going to build a simple but functional tool from scratch. This example demonstrates every concept you'll need for your own tool.

EXAMPLE TOOL: SECURE PASSWORD GENERATOR

This tool generates strong, random passwords based on user preferences. It's simple enough to understand quickly but complex enough to teach all the core concepts.

What This Example Teaches You:

- How to structure an HTML page
- How to handle user input (checkboxes, sliders)
- How to write JavaScript logic
- How to update the page dynamically
- How to add copy-to-clipboard functionality
- How to style with CSS

STEP 1: THE HTML STRUCTURE

HTML provides the skeleton—the elements users interact with. Let's break down each section:

```
<!-- The HTML Structure --> <!DOCTYPE html> <html lang="en"> <head> <!-- Metadata: tells browsers how to display the page --> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <!-- Page title appears in browser tab --> <title>Secure Password Generator</title> </head> <body> <!-- Main container holds all visible content --> <div class="container"> <!-- Header section --> <h1>Password Generator</h1> <p>Create strong, random passwords instantly</p> <!-- Display area for generated password --> <div class="password-display"> <input type="text" id="password" readonly placeholder="Your password will appear here"> <!-- Copy button (we'll add functionality with JavaScript) --> <button id="copyBtn" onclick="copyPassword()"> Copy </button> </div> <!-- Options for password customization --> <div class="options"> <!-- Length slider --> <label> Password Length: <span id="lengthValue">16</span> <input type="range" id="length" min="8" max="64" value="16" oninput="updateLength()"> </label> <!-- Checkboxes for character types --> <label> <input type="checkbox" id="uppercase" checked> Include Uppercase (A-Z) </label> <label> <input type="checkbox" id="lowercase" checked> Include Lowercase (a-z) </label> <label> <input type="checkbox" id="numbers" checked> Include Numbers (0-9) </label> <label> <input type="checkbox" id="symbols" checked> Include Symbols (!@#$%^&*) </label> </div> <!-- Generate button --> <button onclick="generatePassword()" class="generate-btn"> Generate Password </button> </div> <!-- JavaScript file link (we'll create this next) --> <script src="script.js"></script> </body> </html>
```

> *Understanding the HTML*
>
> **Key Concepts:**
>
> - **IDs vs Classes:** IDs (like `id="password"`) are unique identifiers for specific elements. Classes (like `class="container"`) can be reused for styling multiple elements.
> - **Input Types:** The `type` attribute determines how an input behaves. `type="range"` creates a slider, `type="checkbox"` creates a checkbox.
> - **onclick Attribute:** When you click an element with `onclick="functionName()"`, it runs the JavaScript function with that name.
> - **readonly Attribute:** Prevents users from typing in the password field

(they can only copy the generated password).

STEP 2: THE JAVASCRIPT LOGIC (THE BRAIN)

JavaScript is where the magic happens. It reads user preferences, generates the password, and updates the display. Let's build this function by function.

```
// script.js - The complete password generator logic /** * CHARACTER SETS * These strings contain all possible characters for each category * We'll randomly select from these based on user preferences */ const UPPERCASE = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; const LOWERCASE = "abcdefghijklmnopqrstuvwxyz"; const NUMBERS = "0123456789"; const SYMBOLS = "!@#$%^&*()_+-=[]{}|;:,.<>?"; /** * MAIN FUNCTION: generatePassword() * This is called when user clicks "Generate Password" button * It reads all user preferences and creates a password accordingly */ function generatePassword() { // Step 1: Read user preferences from HTML elements const length = document.getElementById("length").value; const useUppercase = document.getElementById("uppercase").checked; const useLowercase = document.getElementById("lowercase").checked; const useNumbers = document.getElementById("numbers").checked; const useSymbols = document.getElementById("symbols").checked; // Step 2: Build available character set based on selections let availableChars = ""; // Start with empty string // If checkbox is checked, add those characters to pool if (useUppercase) availableChars += UPPERCASE; if (useLowercase) availableChars += LOWERCASE; if (useNumbers) availableChars += NUMBERS; if (useSymbols) availableChars += SYMBOLS; // Step 3: Validation - make sure at least one type is selected if (availableChars.length === 0) { alert("Please select at least one character type!"); return; // Exit function early } // Step 4: Generate the password character by character let password = ""; // Loop 'length' times (e.g., if length=16, loop 16
```

STEP 2: THE JAVASCRIPT LOGIC (THE BRAIN)

JavaScript is where the magic happens. It reads user preferences, generates the password, and updates the display. Let's build this function by function.

```
// script.js - The complete password generator logic /** * CHARACTER SETS * These strings contain all possible characters for each category * We'll randomly select from these based on user preferences */ const UPPERCASE = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; const LOWERCASE = "abcdefghijklmnopqrstuvwxyz"; const NUMBERS = "0123456789"; const SYMBOLS = "!@#$%^&*()_+-=[]{}|;:,.<>?"; /** * MAIN FUNCTION: generatePassword() * This is called when user clicks "Generate Password" button * It reads all user preferences and creates a password accordingly */ function generatePassword() { // Step 1: Read user preferences from HTML elements const length = document.getElementById("length").value; const useUppercase = document.getElementById("uppercase").checked; const useLowercase = document.getElementById("lowercase").checked; const useNumbers = document.getElementById("numbers").checked; const useSymbols = document.getElementById("symbols").checked; // Step 2: Build available character set based on selections let availableChars = ""; // Start with empty string // If checkbox is checked, add those characters to pool if (useUppercase) availableChars += UPPERCASE; if (useLowercase) availableChars += LOWERCASE; if (useNumbers) availableChars += NUMBERS; if (useSymbols) availableChars += SYMBOLS; // Step 3: Validation - make sure at least one type is selected if (availableChars.length === 0) { alert("Please select at least one character type!"); return; // Exit function early } // Step 4: Generate the password character by character let password = ""; // Loop 'length' times (e.g., if length=16, loop 16 times) for (let i = 0; i < length; i++) { // Pick a random character from availableChars const randomIndex = Math.floor(Math.random() * availableChars.length); password += availableChars[randomIndex]; } // Step 5: Display the password in the input field document.getElementById("password").value = password; } /** * HELPER FUNCTION: updateLength() * Updates the displayed length value when user moves slider */ function updateLength() { // Get current slider value const length = document.getElementById("length").value; // Update the displayed number next to slider document.getElementById("lengthValue").textContent = length; } /** *
```

```
HELPER FUNCTION: copyPassword() * Copies the generated password to user's clipboard */ function copyPassword() { // Get the password input element const passwordField = document.getElementById("password"); // Check if there's actually a password to copy if (passwordField.value === "") { alert("Generate a password first!"); return; } // Select the password text passwordField.select(); // Copy to clipboard using browser API document.execCommand("copy"); // Give user feedback alert("Password copied to clipboard!"); } // Generate a password immediately when page loads window.onload = function() { generatePassword(); };
```


DEEP DIVE: HOW THE CODE ACTUALLY WORKS

For the Builder: Line-by-Line Explanation

The Character Selection Logic

Let's break down the most critical part—how we build the character pool:

```
let availableChars = ""; if (useUppercase) availableChars += UPPERCASE;
```

What's happening here?

DEEP DIVE: HOW THE CODE ACTUALLY WORKS

For the Builder: Line-by-Line Explanation

The Character Selection Logic

Let's break down the most critical part—how we build the character pool:

```
let availableChars = ""; if (useUppercase) availableChars += UPPERCASE;
```

What's happening here?

1. We start with an empty string: `availableChars = ""`
2. The += operator means "add to the end"
3. So if `useUppercase` is true, we add all 26 uppercase letters
4. If user checks all 4 boxes, `availableChars` becomes a string with 95+ characters

Example: If user selects only uppercase and numbers:

- `availableChars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"` (36 characters)

The Random Selection Algorithm

```
const randomIndex = Math.floor(Math.random() *
availableChars.length); password += availableChars[randomIndex];
```

Breaking this down:

1. `Math.random()` generates a decimal between 0 and 1 (e.g., 0.743)
2. We multiply by `availableChars.length` (e.g., 0.743 × 36 = 26.748)
3. `Math.floor()` rounds down to nearest integer (26.748 → 26)
4. `availableChars[26]` gets the character at position 26
5. We add that character to the password string

Why this is cryptographically sound: `Math.random()` isn't perfect for security, but it's sufficient for most password generators. For maximum security (e.g., generating encryption keys), you'd use `crypto.getRandomValues()` instead.

The Document Object Model (DOM) Connection

```
document.getElementById("password").value = password;
```

What this does:

1. `document` represents the entire HTML page
2. `getElementById("password")` finds the element with `id="password"`
3. `.value` accesses the text inside that input field
4. `= password` sets it to our generated password string

This is how JavaScript communicates with HTML. The browser updates the page in real-time when you change an element's properties.

CUSTOMIZING THIS TEMPLATE

Now that you understand how it works, here's how to adapt this pattern for different tools:

For a different generator tool (e.g., Invoice Number Generator):

1. **Replace character sets** with your generation logic (e.g., "INV-" + year + "-" + sequential number)
2. **Keep the input/checkbox structure** for user customization options
3. **Maintain the display field** for showing results
4. **Keep the copy button** for user convenience

Adding AI Superpowers: OpenAI Integration

Now let's level up. We'll transform our simple password generator into an "AI-Powered Password Strength Analyzer & Suggester." This demonstrates exactly how to integrate AI into any tool.

Adding AI Superpowers: OpenAI Integration

Now let's level up. We'll transform our simple password generator into an "AI-Powered Password Strength Analyzer & Suggester." This demonstrates exactly how to integrate AI into any tool.

THE AI-ENHANCED VERSION

Our upgraded tool will:

1. Generate a password (as before)
2. Analyze its strength using AI
3. Provide specific suggestions for improvement
4. Explain why the password is secure (or isn't)

Why This Adds Value

A basic password generator is a commodity—there are hundreds. But an AI-powered analyzer that explains "This password is strong because it uses uncommon symbol patterns and avoids dictionary words" is differentiated. Users learn while they use your tool.

STEP 1: GETTING YOUR OPENAI API KEY

1. Go to `platform.openai.com`
2. Create an account (free to start)
3. Navigate to API Keys section
4. Click "Create new secret key"
5. Copy the key (it starts with `sk-...`)
6. **Never commit this key to public code repositories**

> *API Key Security*
>
> Your API key is like a password. If exposed publicly, bad actors can rack up charges on your account. For development, store it in an environment variable. For production, use a serverless function that keeps the key server-side (we'll cover this).

STEP 2: THE AI ANALYSIS FUNCTION

```
/** * AI-POWERED PASSWORD ANALYSIS * Sends password to OpenAI for strength evaluation * Returns detailed explanation and suggestions */ async function analyzePasswordWithAI(password) { // Your OpenAI API key (in production, this would be server-side) const API_KEY = "sk-your-api-key-here"; // The API endpoint for chat completions const API_URL = "https://api.openai.com/v1/chat/completions"; // Show loading state to user document.getElementById("analysis").innerHTML = "Analyzing... "; try { // Make the API request const response = await fetch(API_URL, { method: "POST", headers: { "Content-Type": "application/json", "Authorization": `Bearer ${API_KEY}` }, body: JSON.stringify({ model: "gpt-3.5-turbo", // The AI model to use messages: [ { role: "system", content: "You are a cybersecurity expert analyzing password strength. Provide clear, concise feedback." }, { role: "user", content: `Analyze this password: "${password}". Provide: 1) Strength rating (Weak/Moderate/Strong/Very Strong), 2) Specific strengths, 3) Specific improvements if any, 4) Estimated crack time. Keep response under 100 words.` } ], max_tokens: 150, // Limit response length for cost control temperature: 0.7 // Creativity level (0=factual, 1=creative) }) }); // Parse the response
```

```
const data = await response.json(); // Extract the AI's analysis from the response const analysis = data.choices[0].message.content; // Display the analysis to user document.getElementById("analysis").innerHTML = ` <h4>AI Analysis</h4> <p>${analysis}</p> `; } catch (error) { // Handle any errors gracefully console.error("Error analyzing password:", error); document.getElementById("analysis").innerHTML = "Analysis unavailable. Please try again."; } }
```